

USER MANUAL

FOR THE FORTRAN SUBROUTINE

TEM1D

FOR CALCULATION OF

1D TEM RESPONSES AND DERIVATIVES

© 2025 – Niels B. Christensen – nbc@geo.au.dk

A FORTRAN TEM RESPONSE ROUTINE

This is a User Manual for the Open Source FORTRAN subroutine TEM1D for the calculation of TEM responses and derivatives for one-dimensional (1D) models. The paper:

An open source FORTRAN subroutine for calculating TEM responses and derivatives from 1D models

presents the details of the concepts behind the subroutine and its realisation. It is recommended that you read this paper before starting to use the subroutine. In case you use this subroutine in publications, the reference to the paper is:

Christensen N.B., Auken E., Christiansen A.V. and Foged N., 2025. An open source FORTRAN subroutine for calculating TEM responses and derivatives from 1D models. Computers and Geosciences, XX, XX

This manual does not contain references – they can all be found in the paper.

Below is the Introduction from the paper that outlines the considerations behind the program and its conception and realisation. Other sections from the paper are also repeated in this manual.

INTRODUCTION

Presently, several of the commercially available programs for forward calculation, analysis and inversion of transient electromagnetic (TEM) data are bound by restrictions in terms of fees, and proprietary rights of usage and modification. At the same time, there is intense activity in the EM community in the writing of new programs in several different languages, e.g. Python and MATLAB, for the calculation of TEM responses. An asset of these relatively new languages is that they have powerful built-in functions and very efficient commands for vector and matrix manipulations, and they are thus very well suited for writing the inversion part of a TEM inversion program. However, they are often built on an interpretation approach, meaning that, in essence, they are executed one line at a time and not compiled to an executable program. A consequence of this is that functions and subroutines that are called repeatedly will define the overall execution speed of the whole program. One way of getting the best of all worlds is to write a program where the inversion routines are written in a higher order language and where the routines that are called tens of thousands of times are relegated to an external code written in a language that can be compiled. This will overcome the problems with computation time and at the same time permit an inversion formulation in a higher language.

Also in the case of Markov chain Monte Carlo (MCMC) approaches, where forward responses may be needed hundreds of thousands of times or more, it would be advantageous to have an externally compiled code for the forward responses, and in the training of AI networks, rapid forward calculations of training sets are of crucial importance.

All of the situations mentioned above have at least one requirement in common: a fast forward routine for TEM responses, preferably also delivering the derivatives of the response with regard to model parameters to facilitate inversion. Most modern languages are capable of including an external routine as a compiled unit in a different language that will permit fast computation of the forward response and derivatives. An external FORTRAN routine would meet that need. These considerations lie behind the development of a FORTRAN code that can deliver forward responses and derivatives. The code is formulated as a subroutine and is available as ASCII text FORTRAN files. It is up to the user to see to that it is properly compiled to a form that can be integrated with the user's own code. The intent has been to make a subroutine that would cover most of the instrument configurations and modes of application that are in use today, but obviously not everything can be covered. At the same time, it has been a goal to accommodate new developments in TEM instrumentation and modelling, while still being a compact and efficient code.

With regard to integration of the code with other programs, two examples will be mentioned here in more detail. FORTRAN routines can be called from Python programs (<https://numpy.org/doc/stable/f2py/index.html#f2py>) and as an example, the present code should be easily integrated in SimPEG, or other codes like PyGimli (<https://simpeg.xyz/>). Integration of FORTRAN routines in MATLAB is explained in the link: (<https://se.mathworks.com/help/matlab/fortran-language.html>) where a few options are presented. Finally, for computers with modern SSD hard disks, reading from and writing to an ASCII file is almost as fast as if all parameters were present in RAM. This means that "integration for dummies", where e.g. a MATLAB program writes an input file for the FORTRAN program, calls a compiled FORTRAN executable, and reads the output from the FORTRAN program from an ASCII file is not a 'dummy' as it would have been just a few years ago.

The subroutine is delivered as source code written in FORTRAN77, and everyone is welcome to update or change the code according to their particular needs. And to fix bugs, of course, but if you find some, please communicate your findings to the authors.

Many of the recent programming efforts follow a trend that has manifested itself increasingly stronger over the past couple of decades: open source programs. The noble intension behind these efforts is to disseminate computational tools to the rest of the world as a shared resource that can be used, modified. and improved in an open discussion among its users. This is of course of particular importance in the less affluent part of the world where resources are scarce. Several repositories for open source programs exist on

the Internet, such as GitHub, StackExchange and MathWorks, the latter for MATLAB programs. Of particular interest to the geophysical community is the SimPEG repository, an initiative originating from the efforts of scientists from and/or affiliated with University of Vancouver, British Columbia, Canada. This repository aims to have high quality, well documented code with good instructions for its usage. The FORTRAN code presented in this paper can be found at

<https://github.com/hydrogeophysicsgroup/TEM1D>
together with this manual for its use.

PROGRAM STRUCTURE

The program consists of a series of subroutines calculating the response and the derivatives. The program also consists of INCLUDE files containing the COMMON statements of the subroutines. The files and the functions are:

TEM1D.for	The main subroutine to be called by the user.
TEM1DFHT.for	Subroutines for performing the Hankel transform.
TEM1DFUNC.for	A collection of minor subroutines.
TERM1DRESP.for	The response and derivatives routine.
TERM1DRESPIP.for	Same but for a polygonal transmitter (Tx) loop.
TERM1DRESPPOLY.for	Same but for calculations including IP parameters.
TERM1DRESPPOLYIP.for	Same, but for a polygonal Tx and IP parameters.
ARRAYSDIMBL.INC	Contains a PARAMETER statement dimensioning all arrays.
INSTRUMENTBL.INC	Contains the parameters characterising the TEM instrument.
IPBL.INC	Contains the IP parameters – if present.
MODELBL.INC	Contains the parameters describing the 1D model.
POLYGONBL.INC	Contains the parameters describing a polygonal Tx loop.
RESPBL.INC	Contains the response and the derivatives.
WAVEBL.INC	Contains the waveform parameters.

The parameters in the COMMON blocks are explained in the INCLUDE files where they appear.

Besides the basic TEM1D files mentioned above, a driver program is supplied that will read the parameters of the call of TEM1D from a file and write the output from the TEM1D subroutine to a file. This makes it quite easy to set up a test of the program. An executable consisting of the driver program. TEM1DTEST, and the TEM1D subroutine is also supplied.

PROGRAM OPTIONS AND LIMITATIONS

The decisions made in the design considerations are presented in the article mentioned above, but they are repeated here:

(1) The code accommodates the modelling of IP effects.

IP parameters can be included in the forward response, but derivatives with respect to the IP parameters will not be calculated. The IP effect is expressed through a Cole–Cole model and is defined by three additional model parameters for each layer. The IP parameters are easily integrated in the recursive calculation of the kernel function with only a small increase in computation time. Only rarely is brute force inversion carried out on IP parameters. The user can of course calculate numerical derivatives of the IP parameters if needed.

(2) The supported waveform is regarded as a piecewise linear waveform.

With new digital instruments, there is an increasing number of different ways to record and sample the waveform, and it will not be possible to accommodate them all. Besides, presently, practices change quite rapidly. The waveform is treated as a piecewise linear waveform, and if a user puts in a densely sampled waveform with thousands of samples, it will result in a long calculation time. It is recommended that users reduce their waveform definition to fewer samples while still maintaining the accuracy and resolution of the waveform.

(3) Presently the code accommodates the vertical, but not the horizontal field component.

The vertical part of the secondary TEM field is by far the most used for one-dimensional (1D) inversion of TEM data. A horizontal component response might be included later, or, with a limited effort, implemented by a user who needs it.

(4) The code will not include an option of approximate responses.

Computational resources have reached a point where the calculation of accurate responses is not a general issue. If a user wishes to make use of approximate response, their calculation is so fast that it can be included in a calling program.

(5) The code will not include the option of two moments.

Though some TEM instruments make use of two transmitter (Tx) moments, the code will include only one moment. If more than one moment is required, the response routine can be called twice which of course entails a certain computational overhead.

(6) The code does not accommodate integration over gates.

With the advent of digital systems and a dense sampling of the instrument signals, an improved gating has become possible by choosing a smooth,

several times differentiable gating weight function where both its value and several derivatives of the weight function will go to zero at the gate end points, thereby suppressing noise much better than a simple box-car weight function, see e.g. https://en.wikipedia.org/wiki/Window_function. At the moment, there are many approaches to doing this and several different weight functions are in use. It is therefore decided that the code does not deliver an integration over the gates and that this is left up to the user. Most gate integration procedures are quite simple to implement and computationally fast, so there is no point in the code getting in the way of the practices of the various users by forcing a special gate integration on the response. The response and the derivatives are delivered densely sampled in a wide time interval so that the user can implement his/her own gate integration.

CALLING THE SUBROUTINE – INPUT AND OUTPUT PARAMETERS

The main subroutine TEM1D

The main subroutine – the one called by the user – is TEM1D:

```
SUBROUTINE TEM1D (  
# IMLMi, NLAYi, RHONi, DEPNi,  
# IMODIPi, CHAIPi, TAUPI, POWIPi,  
# TXAREAi, RTXRXi, IZEROPOSi,  
# ISHTX1i, ISHTX2i, ISHRX1i, ISHRX2i,  
# HTX1i, HTX2i, HRX1i, HRX2i,  
# NPOLYi, XPOLYi, YPOLYi, XORXi, YORXi,  
# IRESPTYPEi, IDERIVi, IREPi, IWCONVi, NFILTi,  
# REPFREQi, FILTFREQi,  
# NWAVEi, TWAVEi, AWAVEi)
```

The subroutine TEM1D transfers the parameters in the call to the parameters of the COMMON blocks and does a few elementary calculations after which it calls the routine that calculates the response and the derivatives. Finally, the results are written to output files.

If compiled as a .dll to be integrated into another program, or if used with a driver program, the writing to output files must be inactivated and the output parameters included in the call of the TEM1D routine. The output arrays to be included are:

TIMESOUT, RESPOUT0, DRESPOUT

so that the TEM1D subroutine call becomes:

```
SUBROUTINE TEM1D (  
# IMLMi, NLAYi, RHONi, DEPNi,  
# IMODIPi, CHAIPi, TAUIPi, POWIPi,  
# TXAREAi, RTXRXi, IZEROPOSi,  
# ISHTX1i, ISHTX2i, ISHRX1i, ISHRX2i,  
# HTX1i, HTX2i, HRX1i, HRX2i,  
# NPOLYi, XPOLYi, YPOLYi, XORXi, YORXi,  
# IRESPTYPEi, IDERIVi, IREPi, IWCONVi, NFILTi,  
# REPFREQi, FILTFREQi,  
# NWAVEi, TWAVEi, AWAVEi)  
# NTOUT, TIMESOUT, RESPOUT, DRESPOUT)
```

Units

All parameters are in SI units.

All parameters starting with "I" or "N" are INTEGER*4. All others are REAL*8.

The parameters in the call of TEM1D

IMLMi	IMLMi = [0 1] [Few-layer model (FLM) Multi-layer model (MLM)] In an MLM, derivatives are wrt. layer resistivities and Tx height, and the number of model parameters is (NLAY+1), the number of layers plus one. In a FLM, derivatives are wrt. layer resistivities, layer thicknesses and Tx height, and the number of model parameters is 2*NLAY.
NLAYi	The number of model layers.
RHONi	The NLAY layer resistivities in [Ohmm].
DEPNi	The NLAY depths to top of layers. DEPNi(1) = 0, the earth surface. The (NLAY-1) thicknesses are found from the depths.
IMODIPi	IMODIPi = [0 1] [No IP effects included in the response IP included]. If IMODIPi = 0, the next 3 parameter arrays are dummy, but must be present.
CHAIPi	The NLAY chargeabilities. Unit: [V/V].
TAUIPi	The NLAY time constants: Unit: [s].
POWIPi	The NLAY power exponents.
TXAREAi	The area of the Tx.

RTXRXi	The horizontal distance between Tx and Rx.
IZEROPOSi	For IZEROPOSi = 1: If the TX is a polygonal loop, and the Rx is zero-coupled to the Tx, then if the Tx is modelled as a circular loop with the same area as the polygonal loop – in many cases an excellent approximation – the zero-coupled Rx position is found and used in the calculations..
ISHTX1i	[-1 0 1] : Polarity of 1st Tx: [negative does not exist positive].
ISHTX2i	[-1 0 1] : Polarity of 2nd Tx: [negative does not exist positive].
ISHRX1i	[-1 0 1] : Polarity of 1st Rx: [negative does not exist positive].
ISHRX2i	[-1 0 1] : Polarity of 2nd Rx: [negative does not exist positive].
HTX1i	Height of 1st Tx above ground.
HTX2i	Height of 2nd Tx above ground.
HRX1i	Height of 1st Rx above ground.
HRX2i	Height of 2nd Rx above ground.
NPOLYi	The number of apices of the polygonal Tx loop. If NPOLYi = 0, the Tx is circular, and the next 4 parameter (arrays) are dummy, but must be present.
XPOLYi	The x -coordinates of the apices of the polygonal Tx loop.
YPOLYi	The y -coordinates of the apices of the polygonal Tx loop.
XORXi	The x -coordinate of the Rx dipole.
YORXi	The y -coordinate of the Rx dipole.
IRESPTYPEi	[0 1 2] : Response type is [Step Impulse Convolved]. For step and impulse responses, the system response may include filters, if activated, but waveform effects are not modelled. For Convolved response, all system response elements can be modelled.

IDERIVi	[0 1] : [Only model response Model response and derivatives].
IREPi	[0 1] : [Do not Do] model effects of a repeating, alternating waveform.
IWCONVi	[0 1] : Convolution with waveform: [No Yes].
NFILTi	The number of 1st order filters. If NFILTi = 0, no filters will be applied.
REPFREQi	The repetition frequency of the waveform.
FILTFREQi	The cutoff frequencies of the 1st order filters.
NWAVEi	The number samples of the waveform.
TWAVEi	The delay times of the waveform samples.
AWAVEi	The amplitude of the waveform samples.
NTOUT	The number output delay time samples.
TIMESOUT	The output delay times.
RESPOUT	The output response
DRESPOUT	The output derivatives.

NEW UNTRADITIONAL OPTIONS

Two transmitters and two receivers

As can be seen from the previous sections, the subroutine accommodates two Tx's and two Rx's. In recent instrument developments, the use of more than one Tx and one Rx has been seen, and the inclusion of two Tx's and two Rx's makes it possible to model the response from such instruments in the most accurate way in one subroutine call. However, there are a few limitations if more than one Tx or Rx is used:

- Both Tx's must have the same shape and size.
- All possible Tx–Rx pairs must have the same mutual lateral distance between them.

If these conditions are not fulfilled, the subroutine must be called more than once.

IP modelling included

Regarding the option of including IP parameters in the modelling, it shall be repeated here that, if included, IP parameters enters in the forward modelling and in the calculation of derivatives, but derivatives with regard to IP parameters are not supported by the subroutine. The open source code `empymod` also incorporates IP parameters in the forward response.

Custom low pass filters

Modern instruments do not necessarily have a simple low pass filter characteristic consisting of perfect first order and critically damped second order Butterworth filters. The subroutine makes it possible to implement more complicated filter characteristics by defining up to 16 first order filters with a series of cutoff frequencies that together will form the filter. The series expansion expressing the actual filter characteristic, $\hat{F}(s)$, as a product of Laplace transforms of first order Butterworth filters must be found by the user before calling the subroutine by solving a straightforward inversion problem.

$$\hat{F}(s) = \prod_{i=1}^M \frac{1}{[1 + s/(2\pi v_i)]}$$

where s is the Laplace variable and v_i is the cutoff frequency of the i 'th first order filter. A standard critically damped second order filter can be realised as the product of two first order filters with the same cutoff frequency.

INTEGRATION OF THE TEM1D ROUTINES IN EXTERNAL PROGRAMS

The TEM1D subroutine can be integrated in and called from other programs in several different ways. Below are a few considerations regarding this process.

Integration in another FORTRAN program

The most simple and straightforward situation is of course to integrate the TEM1D FORTRAN subroutine in a FORTRAN program. All that is needed is to compile the external FORTRAN program files plus the TEM1D FORTRAN files together to form an executable file.

In this case, the output parameters NTOUT, RESPOUT, and DRESPOUT should be included in the call of the TEM1D subroutine. The lines of the TEM1D code writing to external files can be deleted / inactivated or not, according the user's wishes.

Integration in higher language programs

The integration of the TEM1D routines in programs written in higher languages, e.g. MATLAB or python, must be done according to the special requirements of those languages. This most often entails a special compilation producing a .dll file and sometimes the inclusion of special statements in both the calling code and the TEM1D code. Instructions in how to do this in the best way can be found in the help files of the specific language manuals. A few of them are cited in the paper.

Also in this case, the output parameters NTOUT, RESPOUT, and DRESPOUT should be included in the call of the TEM1D subroutine. The lines of the TEM1D code writing to external files can be deleted / inactivated or not, according the user's wishes.

Integration for 'dummies'

First of all, this headline is not at all meant in a derogatory sense. I have actually practiced this method myself to make life easier – for once – and it is better than it sounds.

In this approach, the calling program writes an ASCII file with the necessary input parameters to the hard drive. Subsequently the external program, calls a FORTRAN driver program that reads the input file and then calls the TEM1D subroutine. When calculations are finished, the results are transferred back to the FORTRAN driver program and that program writes the output to an ASCII file – which is then read by the external calling program. With the advent of SSD hard drives, writing and reading from the disk is almost as fast as manipulations inside the RAM, so very little time is lost by this approach. It does require addressing the operating system for every call, but again – little time is wasted in this process.

In the open source program package, we have included a FORTRAN executable that consists of the driver program, TEMTEST, compiled together with the TEM1D subroutines. This program offers an 'integration for dummies' approach which may be the easiest way to test out the TEM1D

package. The input format of the driver program is given below in a pseudo-language. Note that if a flag parameter indicating if an option is activated or not is zero, then the pertaining parameters are not written to the input file. Also notice that a text line is written as a header for each of the input blocks. This makes the file more readily readable and it becomes easier to find errors.

The ASCII input file – FORREAD – for the driver program

```
%=====
% --- WRITE TO FILE: FORREAD.
%=====

%-----
% --- Writing Model Parameters.
%-----
>>> Write text string for the model parameters group.
    '>> MODEL: IMLM,NLAY, (RHON(I),DEPN(I),I=1,NLAY) '

write IMLM
write NLAY
write RHON(1), DEPN(1)
write RHON(2), DEPN(2)
    . . .
write RHON(NLAY), DEPN(NLAY)

%-----
% --- Writing IP Parameters.
%-----
>>> Write text string for the IP parameters group.
    '>> IP PARAMETERS: IMODIP, (CHAIP(I),TAUIP(I),POWIP(I),I=1,NLAY) '

write IMODIP
IF IMODIP > 0
    write CHAIP(1),TAUIP(1),POWIP(1)
    write CHAIP(2),TAUIP(2),POWIP(2)
    . . .
    write CHAIP(NLAY),TAUIP(NLAY),POWIP(NLAY)
end IF block

%-----
% --- Writing Instrument Parameters.
%-----
>>> Write text string for the Instrument Parameters
    '>> INSTRUMENT:
        TXAREA,ITX1,IRX1,ITX2,IRX2,HTX1,HRX1,HTX2,RTX2,RTXRX,IZEROPOS '

write TXAREA
write ITX1
write IRX1
write ITX2
write IRX2
write HTX1
write HRX1
write HTX2
write RTX2
write RTXRX
write IZEROPOS
```

```
%-----
% --- Writing Polygonal Tx Loop Parameters
%-----
>>> Write text string for the Instrument Parameters
    '>> POLYGONAL TRANSMITTER:
      NPOLY, (XPOLY(I),YPOLY(I),I=1,NPOLY),XORX,YORX,') ;

write NPOLY
IF NPOLY > 0, WRITE THE POLYGONAL LOOP PARAMETERS
  write XPOLY(1),YPOLY(1)
  write XPOLY(2),YPOLY(2)
  . . .
  write XPOLY(NPOLY),YPOLY(NPOLY)

  write XORX
  write YORX

end IF block

%-----
% --- Writing Response Parameters
%-----
>>> Write text string for the Response Parameters
    '>> RESPONSE PARAMETERS: IRESPTYPE,IDERIV,IREFMOD,REFREQ,
      NFILT, (FILT(I),I=1,NFILT),NWAVE, (TWAVE(I),AWAVE(I),I=1,NWAVE) '

write IRESPTYPE
write IDERIV

write IREFMOD
IF IREFMOD > 0
  write REFREQ
end IF block;

write NFILT
IF NFILT > 0;
  write FILTFREQ(1)
  write FILTFREQ(2)
  . . .
  write FILTFREQ(NFILT)
end IF block;

write NWAVE
IF NWAVE > 0
  write (TWAVE(1),AWAVE(1)
  write (TWAVE(2),AWAVE(2)
  write (TWAVE(NWAVE),AWAVE(NWAVE)
end IF block;
```

Below is a commented example of the input file FORREAD for the TEM1DTEST driver program:

```
>> MODEL: IMLM,NLAY, (RHON(I),DEPN(I),I=1,NLAY)
1          > Modelling is done assuming a multi-layer model.
3          > A 3-layer model.
30.00      0.          > Resistivity and depth to top of 1st layer.
200.00     10.         > Resistivity and depth to top of 2ndt layer.
5.00       50.         > Resistivity and depth to top of 3rd layer.
>> IP PARAMETER: IMLM,NLAY, (RHON(I),DEPN(I),I=1,NLAY)
0          > No IP modelling.
>> INSTRUMENT: TXAREA,ITX1,IRX1,ITX2,IRX2,HTX1,HRX1,HTX2,RTX2,RTXRX,IZEROPOS
8.41       > Area of the TX loop.
1          > Tx #1 is activated.
1          > Rx #1 is activated.
0          > Tx #2 is NOT activated.
0          > Rx #2 is NOT activated.
0.97       > Tx height.
0.30       > Rx height.
0.00
0.00
9.40       > The lateral Tx-Rx distance.
0          > No zero positioning of Rx.
>> POLYTX: NPOLY, (XPOLY(I),YPOLY(I),I=1,NPOLY),X0RX,Y0RX,Z0RX
0          > The Tx is modelled as a circualr loop.
>> RESPONSE: IRESPTYPE,IDERIV,IREFMOD,REPFREQ,NFILT,(FILT(I),I=1,NFILT)
2          > System convolved response.
0          > Forward modelling; no derivatives.
0          > No modelling of repetition effects.
2          > Two 1st order low-pass filters.
450000     > The cutoff frequency of the 1st filter.
800000     > The cutoff frequency of the 2nd filter.
18         > Number of waveform samples.
-2.0000e-04 0.0000    > The times and amplitudes of the waveform.
-1.7500e-04 0.2558    . . .
-1.5000e-04 0.4551    . . .
-1.2500e-04 0.6102
-1.0000e-04 0.7311
-7.5000e-05 0.8252
-5.0000e-05 0.8985
-2.5000e-05 0.9556
0.0000e+00 1.0000
1.0000e-06 0.2154
2.0000e-06 0.0464
3.0000e-06 0.0100
4.0000e-06 0.0022
4.1000e-06 0.0018
4.2000e-06 0.0016
4.3000e-06 0.0014
4.4000e-06 0.0012
4.5000e-06 0.0000
```

The ASCII output file – FORWRITE – from the driver program

Output from the driver program, is written to the file, FORWRITE, in the format shown below for the case of a Multi-layer model with 13 layers:

```

Times:      1.0000E-08   1.2589E-08   1.5849E-08   ...   3.1623E-03   3.9811E-03   5.0119E-03
Response:   -7.0473E-06  -1.1821E-05  -1.9592E-05  ...   4.2880E-16   2.6534E-16   7.5726E-17
Deriv_1:    -3.0802E-04  -5.3203E-04  -9.0345E-04  ...  -2.7167E-15   3.0848E-15  -6.9275E-16
Deriv_2:    -9.8449E-06  -2.2911E-05  -5.0375E-05  ...   1.4932E-15   8.0996E-16  -1.0536E-16
            -2.8684E-07  -1.0334E-06  -3.3092E-06  ...   1.7918E-15   1.8103E-16  -1.7427E-16
            2.5154E-11  -6.4971E-10  -6.5970E-09  ...  -4.3898E-16   2.3283E-16   7.7277E-17
            1.6623E-10   1.5745E-10   1.2861E-10  ...   1.7585E-15   2.1866E-16   1.8375E-16
            2.3837E-10   2.3852E-10   2.4075E-10  ...   1.9695E-15   4.9494E-16   1.0851E-16
            3.5983E-10   3.5999E-10   3.6016E-10  ...   3.6923E-16   1.3699E-15   1.0672E-16
            5.0101E-10   5.0118E-10   5.0140E-10  ...   6.7395E-16   7.6616E-16   4.6271E-16
            7.0697E-10   7.0715E-10   7.0737E-10  ...   3.3711E-15   6.7888E-16   2.7770E-16
            9.7649E-10   9.7666E-10   9.7686E-10  ...   1.4563E-15   8.6547E-16   5.7817E-16
            1.3145E-09   1.3146E-09   1.3148E-09  ...   3.3184E-15   1.3807E-15   2.9166E-16
            1.7128E-09   1.7129E-09   1.7131E-09  ...   2.8623E-15   1.3948E-15   5.2694E-16
Deriv_13:   5.5390E-08   5.5389E-08   5.5389E-08  ...   3.3129E-14   1.5856E-14   2.8772E-14
Deriv-HTX:  3.2987E-05   5.3816E-05   8.6753E-05  ...  -3.2270E-16  -8.9783E-17   1.1491E-17

```

Each row of the above block contains NTIMES values.

For a MLM with NLAY layers, where only layer conductivities are model parameters, the sequence of derivatives is $dR/d\sigma_i, i = 1, \dots, \text{NLAY}$. For FLM, where both conductivities and layer thicknesses are model parameters, the sequence is first the derivatives with regard to conductivities from the top and down, then the derivatives with regard to layer thicknesses numbered from the top and down. For both model types, the derivative of the response with regard to Tx height is the last of the sequence. Notice that the derivatives with regard to conductivities are always linear derivatives, i.e. derivatives of the linear response with regard to linear conductivities. The number of derivatives, i.e. the number of inversion parameters, NPARM, is therefore (NLAY+1) for MLMs and 2 · NLAY for FLMs, respectively.

The 1D array of output delay times is dimensioned as TIMESOUT(1:NTIMES) and the 1D array of responses is dimensioned as RESPOUT0(1:NTIMES), while the 2D array of derivatives is dimensioned as DRESPOUT(1:NTIMES,1:NPARM), where NTIMES is the number of time samples and NPARM is the number of inversion parameters. All output values are normalised to a Tx moment of 1.

The output file is in XYZ format: The same number of columns in each line. It should be straightforward to read or load the file into the calling program.

FINAL REMARKS

As mentioned earlier, the integration over gates is not included in the program. The output from the program is a densely sampled array of values of the response and derivatives for delay times from 10 ns to the end of the off-time which depends on the repetition frequency. Sampling density is hardwired to 10 per decade. This is enough for a sufficiently accurate interpolation when the integration of gates is implemented outside of this program.

Notice that the ultra-early delay times may be inconsistent with the assumption that the quasi-static approximation is valid. Depending on the instrument height the earliest delay time that can be considered valid is given by $(H^{Tx} + H^{Rx})/c_0$ where H^{Tx} is the Tx height, H^{Rx} is the Rx height, and c_0 is the speed of light in vacuum. This means that for an airborne system at a height of 30 m above the ground, the limiting delay time would be ≈ 200 ns. If the ultra early times are relevant for the instrument and the measurement strategy – and with present technology that is still unlikely – a full relativistic modelling of the EM responses would be necessary.
